
ENSEM

Ecole Nationale Supérieure d'Electricité et de Mécanique
Université Hassan II de Casablanca

Project AeroTrack

*Securing Airplane Spare Parts Provenance
using Ethereum Smart Contracts*

Module: Blockchain Tools & Applications

Semester: S4 – 2025/2026

Student: Noaman Makhlouf

Supervisor: Prof. Blockchain Engineering Lab

May 2026

Contents

1	Introduction	2
1.1	Industrial Context	2
1.2	The Blockchain Solution	2
1.3	Lab Objectives	2
2	Technical Background	2
2.1	Ethereum and Smart Contracts	2
2.2	Key Solidity Concepts	3
3	Implementation	3
3.1	Step 1: Data Architecture	3
3.2	Step 2: Role-Based Access Control	4
3.3	Step 3: Core Business Functions	5
3.3.1	Manufacturing a Part	5
3.3.2	Transferring Ownership	6
3.4	Step 4: Events and Traceability	8
3.5	Step 5: Enhancement – Certified Manufacturer Registry	9
3.6	Step 6: Decentralized Identifiers (DIDs)	12
3.7	Step 7: IoT Oracle Simulation	16
3.8	Step 8: Real IoT Oracle with MQTT Broker	20
4	Conclusion	23
A	Complete Source Code	25
A.1	Step 1–4: Base AeroTrack Contract	25
A.2	Step 5: Certified Manufacturer Registry	26
A.3	Step 6: Decentralized Identifiers (DIDs)	28
A.4	Step 7: IoT Oracle Simulation	30

1 Introduction

1.1 Industrial Context

In the aviation industry, **dependability and safety** are paramount. A single counterfeit or poorly maintained spare part can compromise the integrity of an entire aircraft. Traditional supply chains rely on centralized databases and paper-based certificates (such as the EASA Form 1 or FAA 8130-3), which are susceptible to forgery, loss, and data manipulation.

1.2 The Blockchain Solution

By leveraging **Ethereum smart contracts**, we can create an immutable, transparent, and decentralized ledger for tracking the lifecycle of airplane parts. From manufacturing through ownership transfers to maintenance logging, every action is cryptographically secured and permanently recorded.

1.3 Lab Objectives

This project aims to:

- Understand the role of blockchain in industrial supply chain security
- Apply Solidity concepts (Structs, Mappings, Modifiers, Events) to a real-world problem
- Implement Role-Based Access Control (RBAC)
- Deploy, test, and debug a smart contract using Remix IDE

2 Technical Background

2.1 Ethereum and Smart Contracts

Ethereum is a decentralized platform that enables developers to deploy self-executing programs called **smart contracts**. Once deployed, a smart contract's code cannot be modified, ensuring trust and transparency.

2.2 Key Solidity Concepts

Concept	Description
State Variables	Data stored permanently on the blockchain
Structs	Custom data types grouping related fields
Mappings	Hash-table-like structures for O(1) key-value access
<code>msg.sender</code>	The Ethereum address calling the current function
Modifiers	Reusable precondition checks attached to functions
Events	Cheap log entries for off-chain traceability

Table 1: Core Solidity concepts used in AeroTrack

3 Implementation

3.1 Step 1: Data Architecture

We define the data model using an **enum** for part lifecycle states and a **struct** for part attributes.

```

1  enum PartStatus { Manufactured, InTransit, Installed, Retired }
2
3  struct Part {
4      uint256 serialNumber;
5      string partName;
6      address manufacturer;
7      address currentOwner;
8      PartStatus status;
9      bool exists; // Security flag
10 }
```

Listing 1: Part status enum and struct definition

The `exists` boolean is a security pattern: since Solidity mappings return default zero-values for non-existent keys, this flag distinguishes between a real part and an empty struct.

The **mapping** provides instant access to any part by its serial number:

```

1  address public regulatoryAuthority;
2  mapping(uint256 => Part) public parts;
```

Listing 2: State variables

After compiling and deploying the contract skeleton on Remix VM, the deployment succeeds as shown in Figure 1.

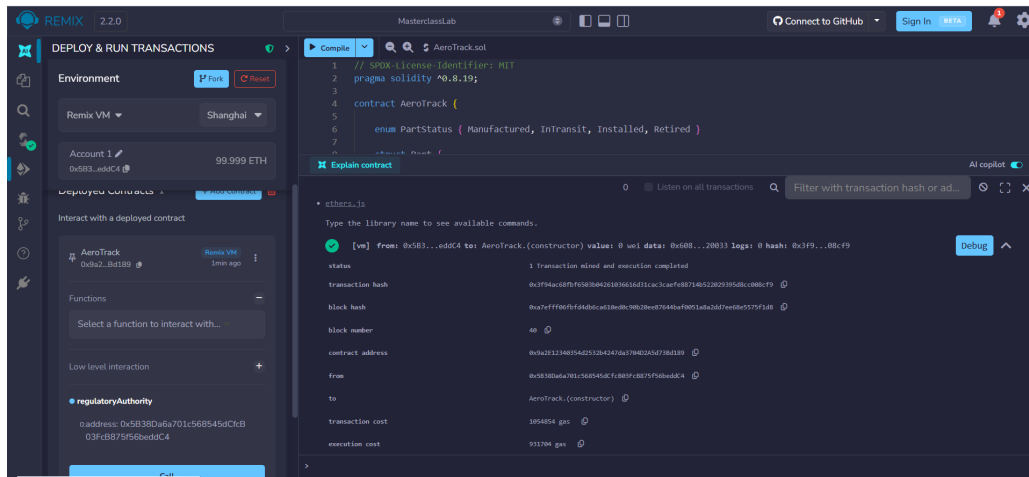


Figure 1: Successful deployment of the AeroTrack contract on Remix VM

Calling the `regulatoryAuthority` getter confirms that the deployer's address is correctly stored (Figure 2).

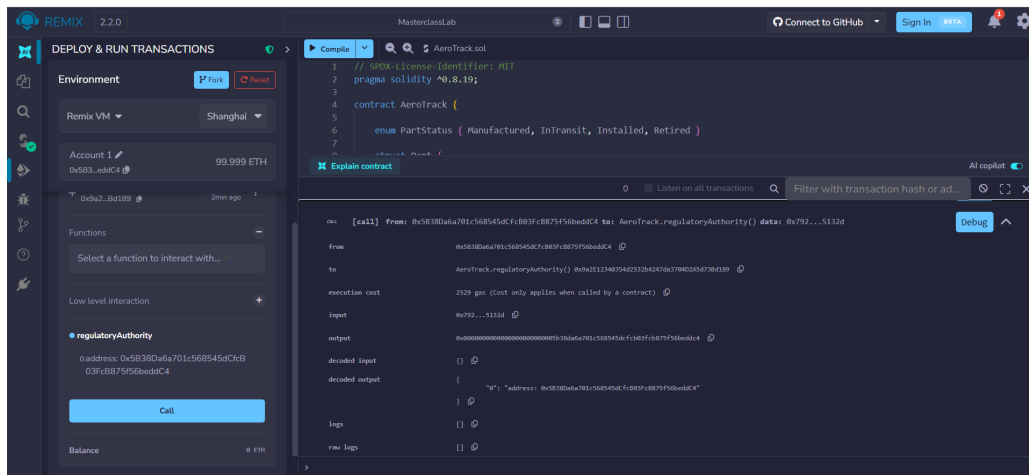


Figure 2: Querying regulatoryAuthority returns the deployer's address

3.2 Step 2: Role-Based Access Control

Security is enforced through **modifiers** – reusable checks that execute before the function body.

```

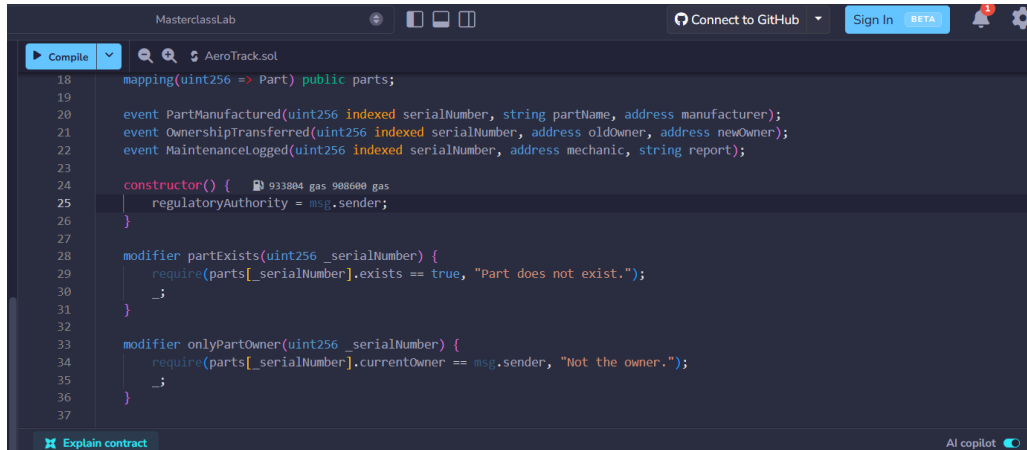
1  modifier partExists(uint256 _serialNumber) {
2      require(parts[_serialNumber].exists == true,
3          "Part does not exist.");
4      _;
5  }
6
7  modifier onlyPartOwner(uint256 _serialNumber) {
8      require(parts[_serialNumber].currentOwner == msg.sender,
9          "Not the owner.");
10     _;
11 }

```

Listing 3: Security modifiers

The `_;` placeholder indicates where the function body is inserted after the checks pass. If `require()` fails, the entire transaction reverts.

Figure 3 shows the modifiers implemented in Remix, and Figure 4 confirms successful compilation.

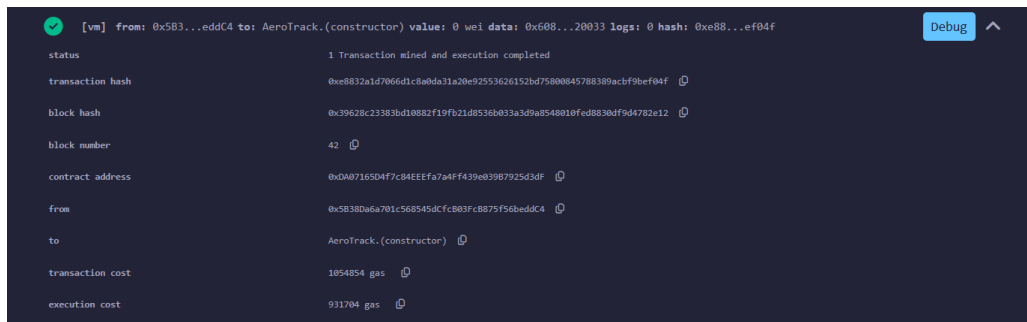


```

18 mapping(uint256 => Part) public parts;
19
20 event PartManufactured(uint256 indexed serialNumber, string partName, address manufacturer);
21 event OwnershipTransferred(uint256 indexed serialNumber, address oldOwner, address newOwner);
22 event MaintenanceLogged(uint256 indexed serialNumber, address mechanic, string report);
23
24 constructor() {
25     regulatoryAuthority = msg.sender;
26 }
27
28 modifier partExists(uint256 _serialNumber) {
29     require(parts[_serialNumber].exists == true, "Part does not exist.");
30     _;
31 }
32
33 modifier onlyPartOwner(uint256 _serialNumber) {
34     require(parts[_serialNumber].currentOwner == msg.sender, "Not the owner.");
35     _;
36 }
37

```

Figure 3: Role-based access control modifiers in the Remix editor



```

[vm] from: 0x5B3...eddC4 to: AeroTrack.(constructor) value: 0 wei data: 0x608...20033 logs: 0 hash: 0xe88...ef04f
status 1 Transaction mined and execution completed
transaction hash 0xe8832a1d7066d1c8a0da31a20e92553626152bd75800845788389acbf9bef04f
block hash 0x39628c23383bd10882f19fb21d8536b03a3d9a8548010fed8830df9d4782e12
block number 42
contract address 0xD40716504f7c84EEef7a4FFA39e03987925d3dF
from 0x5B38D06a701c568545dCfC803fC8B75f56beddC4
to AeroTrack.(constructor)
transaction cost 1854854 gas
execution cost 931704 gas

```

Figure 4: Successful compilation after adding security modifiers

3.3 Step 3: Core Business Functions

3.3.1 Manufacturing a Part

```

1 function manufacturePart(uint256 _serialNumber,
2                           string memory _partName) public {
3     require(parts[_serialNumber].exists == false,
4             "Serial Number taken!");
5
6     parts[_serialNumber] = Part({
7         serialNumber: _serialNumber,
8         partName: _partName,
9         manufacturer: msg.sender,
10        currentOwner: msg.sender,
11        status: PartStatus.Manufactured,
12        exists: true
13    });
14
15    emit PartManufactured(_serialNumber, _partName, msg.sender);

```

16 } }

Listing 4: manufacturePart function

3.3.2 Transferring Ownership

```

1 function transferPart(uint256 _serialNumber, address _newOwner)
2     public partExists(_serialNumber) onlyPartOwner(_serialNumber)
3 {
4     require(_newOwner != address(0), "Invalid address.");
5
6     address oldOwner = parts[_serialNumber].currentOwner;
7     parts[_serialNumber].currentOwner = _newOwner;
8     parts[_serialNumber].status = PartStatus.InTransit;
9
10    emit OwnershipTransferred(_serialNumber, oldOwner, _newOwner);
11 }

```

Listing 5: transferPart function

Figures 5 to 9 illustrate the complete workflow of manufacturing, transferring, and testing access control.

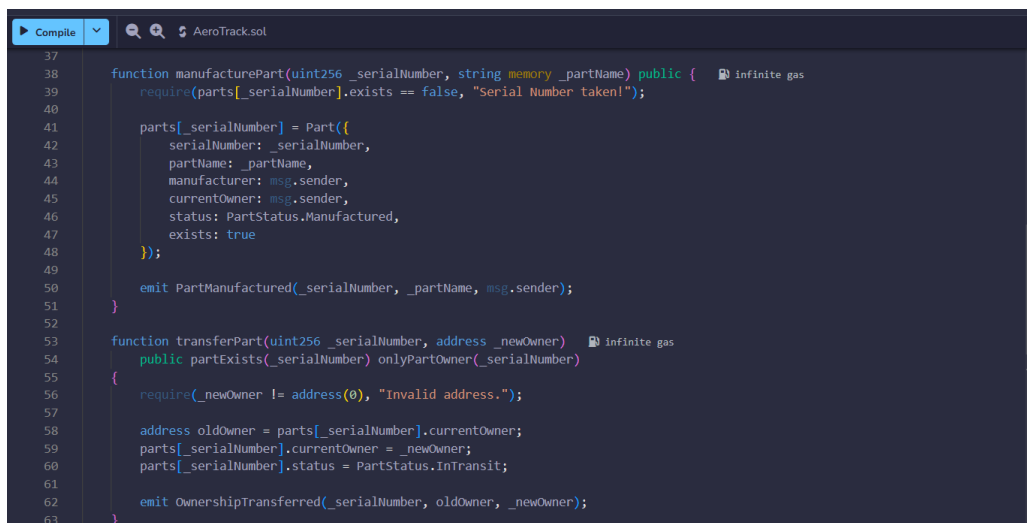


Figure 5: Core functions code in the Remix editor

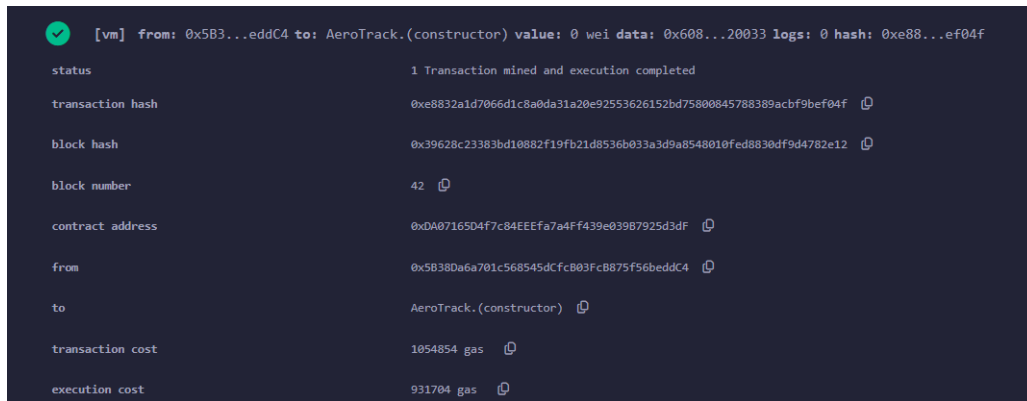


Figure 6: Successful deployment of the contract with core functions

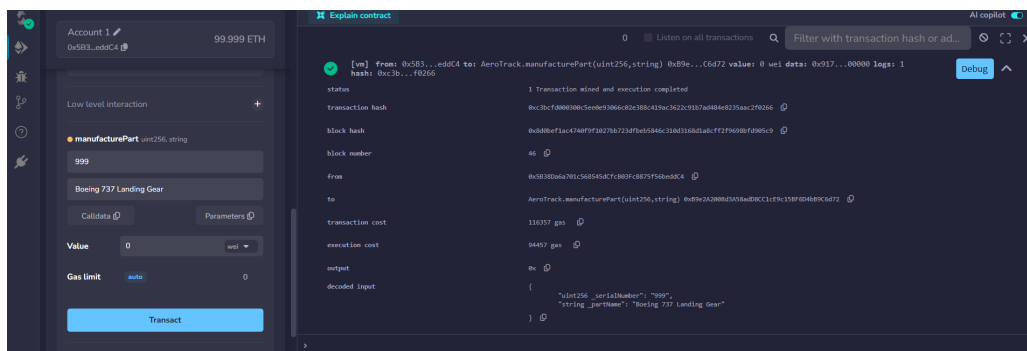


Figure 7: Calling manufacturePart to register a new part (serial 999)

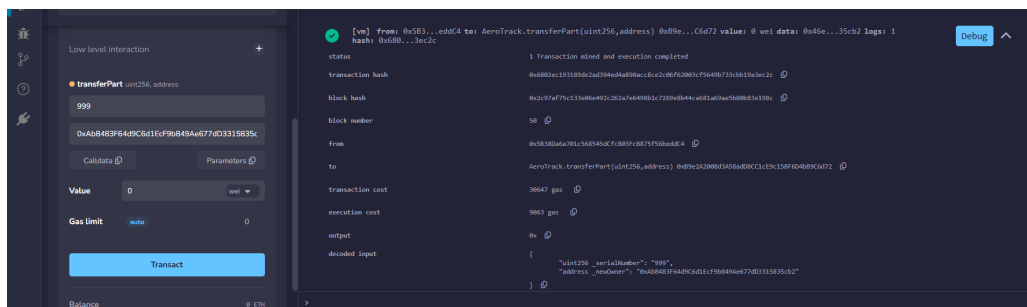


Figure 8: Successful ownership transfer from manufacturer to airline

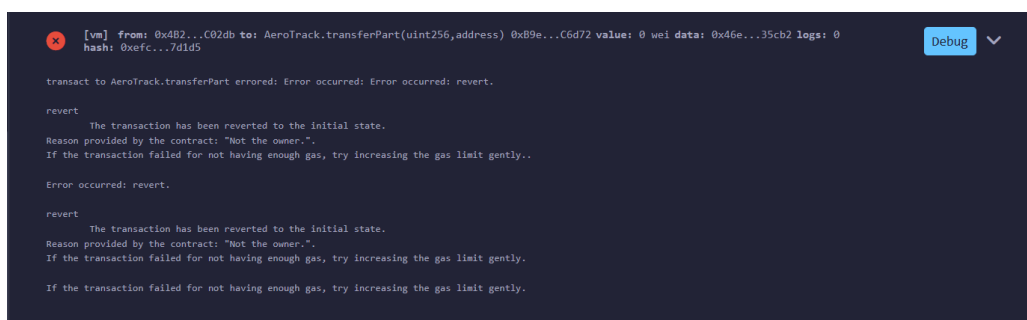


Figure 9: Security test: unauthorized transfer attempt reverts with “Not the owner”

3.4 Step 4: Events and Traceability

Events provide a gas-efficient way to log historical data. They are written to transaction logs (not contract storage), making them approximately 10–100x cheaper.

```

1 event PartManufactured(uint256 indexed serialNumber,
2                       string partName, address manufacturer);
3 event OwnershipTransferred(uint256 indexed serialNumber,
4                             address oldOwner, address newOwner);
5 event MaintenanceLogged(uint256 indexed serialNumber,
6                           address mechanic, string report);
7
8 function logMaintenance(uint256 _serialNumber,
9                           string memory _report)
10 public partExists(_serialNumber) onlyPartOwner(_serialNumber)
11 {
12     emit MaintenanceLogged(_serialNumber, msg.sender, _report);
13     parts[_serialNumber].status = PartStatus.Installed;
14 }

```

Listing 6: Event declarations and maintenance function

The `indexed` keyword allows external applications to efficiently filter events by serial number.

Figures 10 to 14 demonstrate the events in action.

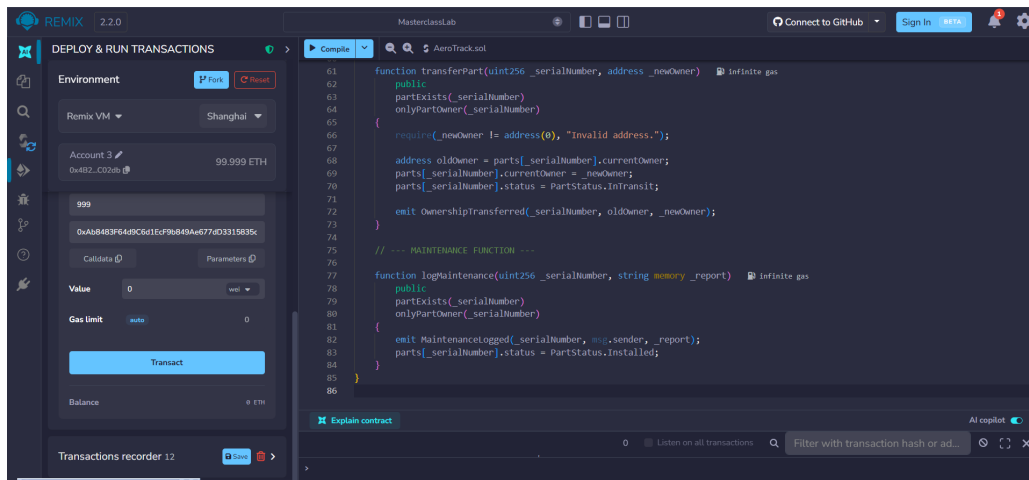


Figure 10: Contract code with event declarations and `logMaintenance` function

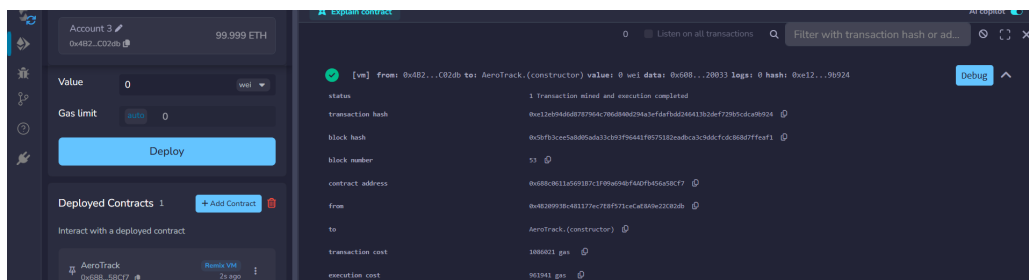


Figure 11: Successful deployment of the contract with events

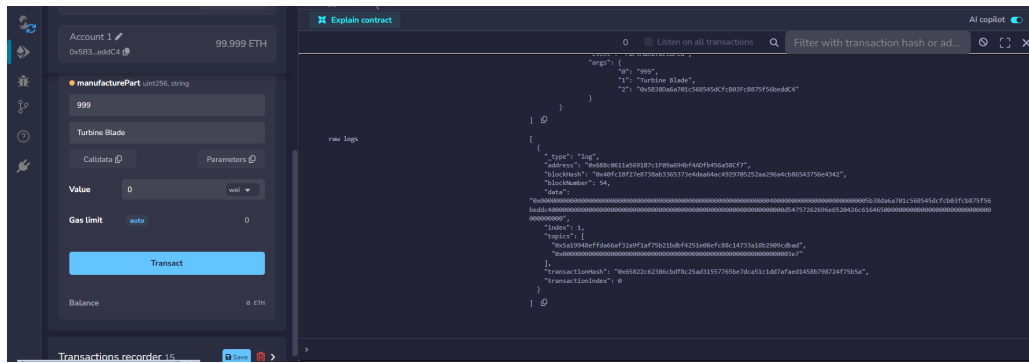


Figure 12: Transaction logs showing the PartManufactured event

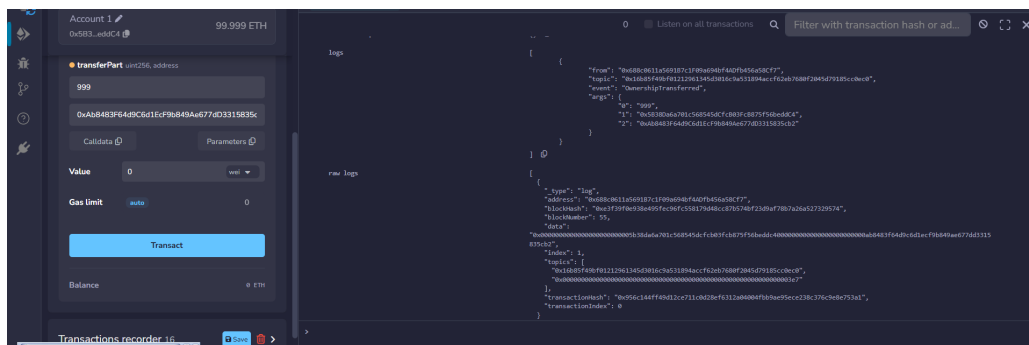


Figure 13: Transaction logs showing the OwnershipTransferred event

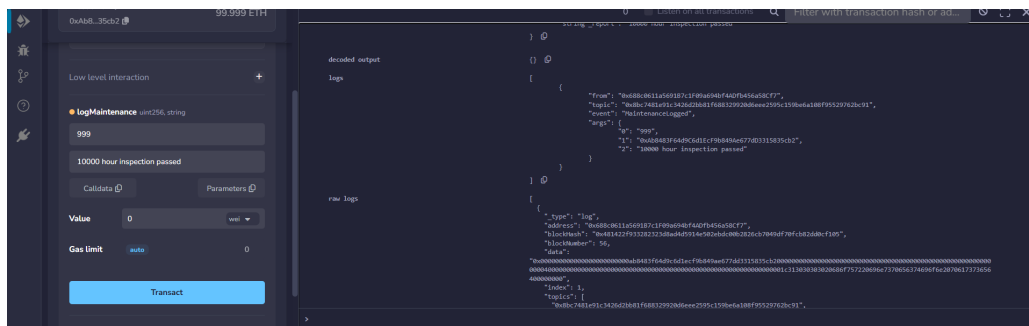


Figure 14: Calling logMaintenance – event emitted with maintenance report

3.5 Step 5: Enhancement – Certified Manufacturer Registry

To address the **Decentralized Identifiers (DIDs)** topic from the lab's future research section, we implemented a manufacturer certification system. The **regulatoryAuthority** (FAA/EASA) maintains a registry of approved manufacturers. Only certified addresses can call `manufacturePart`.

```

1 mapping(address => bool) public certifiedManufacturers;
2
3 modifier onlyAuthority() {
4     require(msg.sender == regulatoryAuthority, "Not the authority.");
5     _;
6 }

```

```

7
8 modifier onlyCertified() {
9     require(certifiedManufacturers[msg.sender],
10         "Not a certified manufacturer.");
11     _;
12 }
13
14 function certifyManufacturer(address _manufacturer)
15     public onlyAuthority {
16     certifiedManufacturers[_manufacturer] = true;
17     emit ManufacturerCertified(_manufacturer);
18 }
19
20 function revokeManufacturer(address _manufacturer)
21     public onlyAuthority {
22     certifiedManufacturers[_manufacturer] = false;
23     emit ManufacturerRevoked(_manufacturer);
24 }

```

Listing 7: Manufacturer certification system

The `manufacturePart` function now includes the `onlyCertified` modifier, preventing uncertified addresses from registering parts.

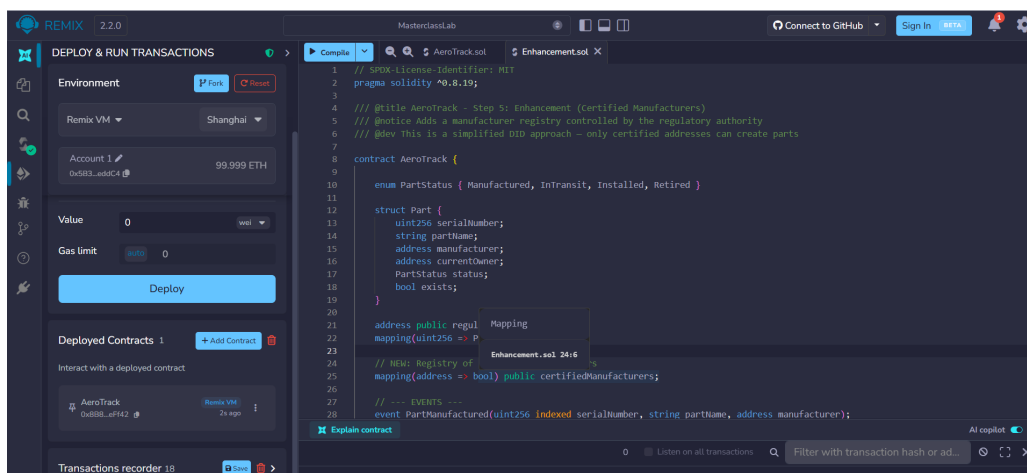


Figure 15: Enhanced contract with manufacturer certification system



Figure 16: Manufacturing attempt fails without certification

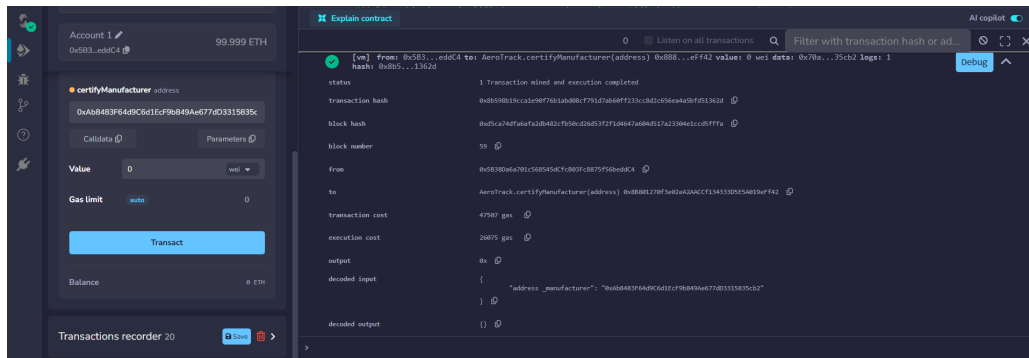


Figure 17: Authority certifies Account 2 as a manufacturer

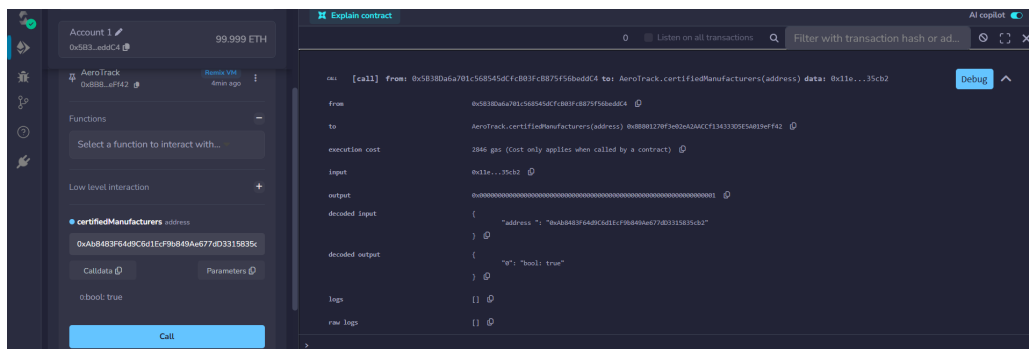


Figure 18: Verifying certification status returns true

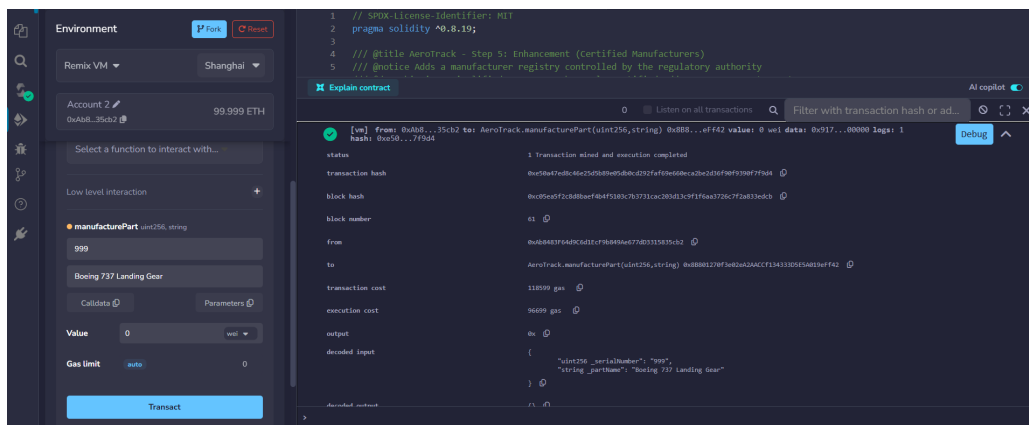


Figure 19: Certified Account 2 successfully manufactures a part

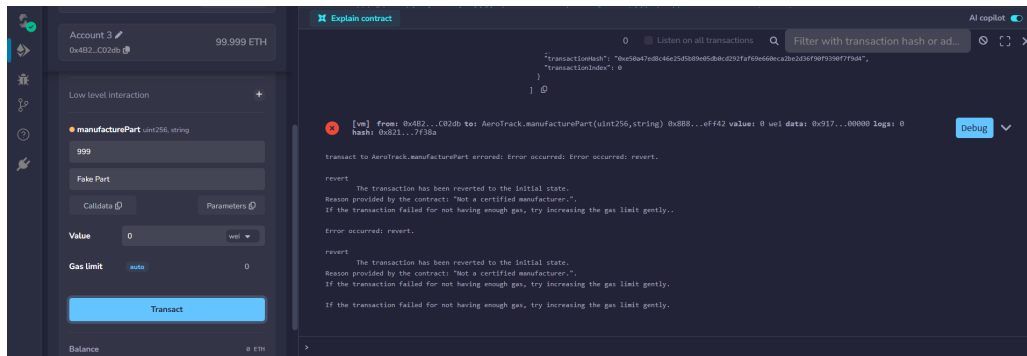


Figure 20: Uncertified Account 3 fails to manufacture

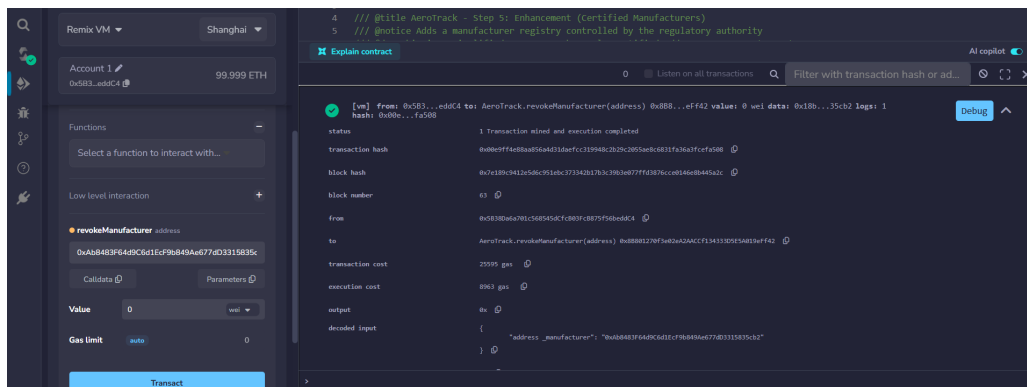


Figure 21: Authority revokes Account 2's certification

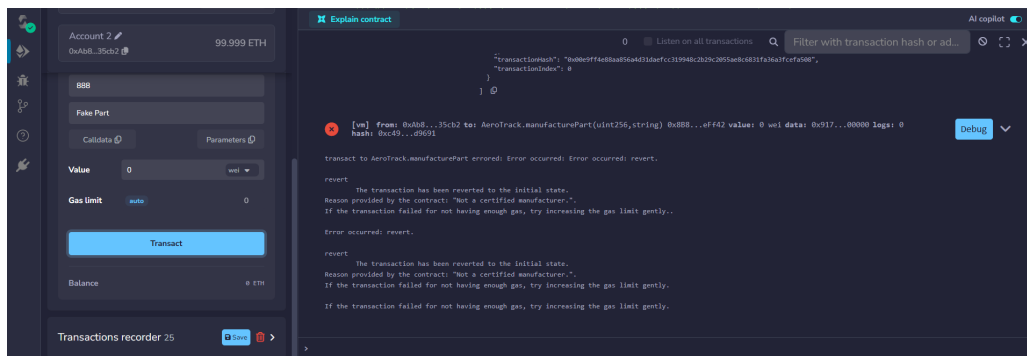


Figure 22: Account 2 fails to manufacture after revocation

3.6 Step 6: Decentralized Identifiers (DIDs)

To fully address the DID research question, we extended the certification system with **on-chain identity storage**. Instead of a simple boolean, the authority now registers a manufacturer's full identity: company name, country, and official certification number.

```
1 struct ManufacturerIdentity {
2     string name; // e.g., "Boeing"
3     string country; // e.g., "USA"
4     string certificationId; // e.g., "FAA-2024-001"
5     bool isActive;
```

```

6      uint256 registeredAt;          // Block timestamp
7  }
8
9  mapping(address => ManufacturerIdentity)
10     public manufacturerIdentities;

```

Listing 8: ManufacturerIdentity struct (DID)

The `registerManufacturer` function stores the full identity and certifies the address simultaneously:

```

1  function registerManufacturer(
2      address _manufacturer,
3      string memory _name,
4      string memory _country,
5      string memory _certificationId
6  ) public onlyAuthority {
7      manufacturerIdentities[_manufacturer] = ManufacturerIdentity({
8          name: _name,
9          country: _country,
10         certificationId: _certificationId,
11         isActive: true,
12         registeredAt: block.timestamp
13     });
14     certifiedManufacturers[_manufacturer] = true;
15     emit ManufacturerRegistered(_manufacturer, _name,
16         _certificationId);
17 }

```

Listing 9: DID registration function

This answers the lab’s research question: anyone can query `manufacturerIdentities[address]` to cryptographically verify that an address belongs to a specific company.

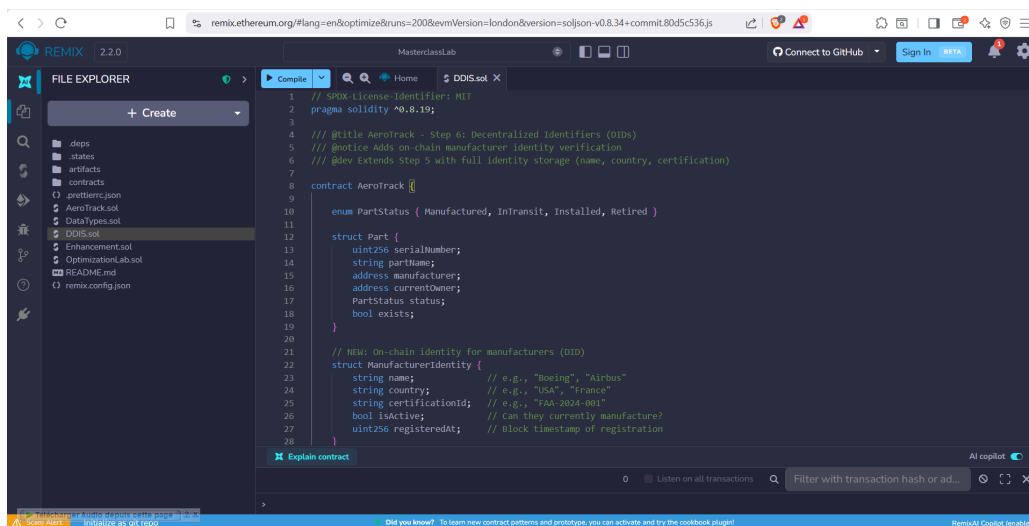


Figure 23: DID-enhanced contract code in Remix

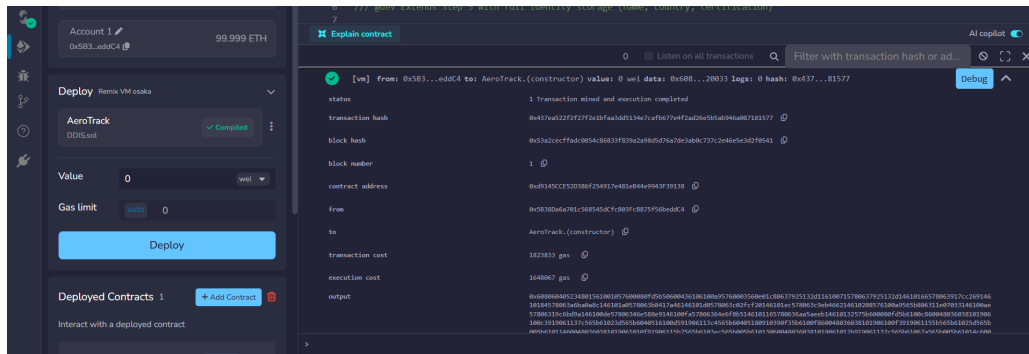


Figure 24: Successful deployment of the DID-enhanced contract

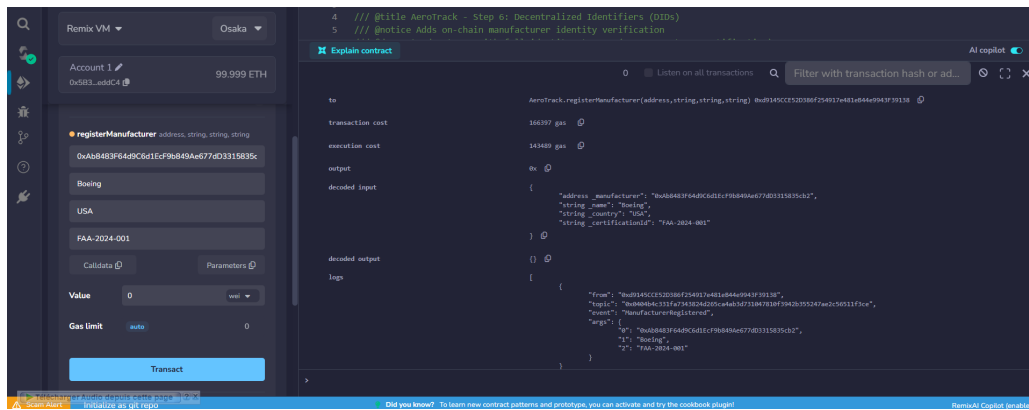


Figure 25: Authority registers Boeing with full identity (name, country, cert ID)

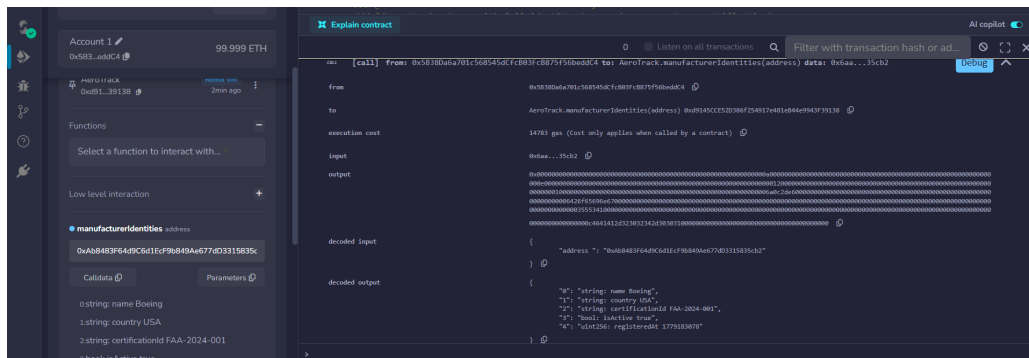


Figure 26: Querying manufacturerIdentities returns full on-chain identity

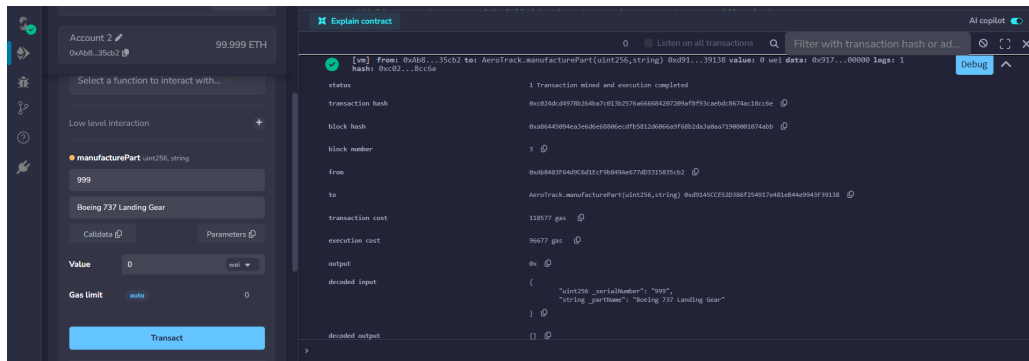


Figure 27: Registered manufacturer successfully creates a part

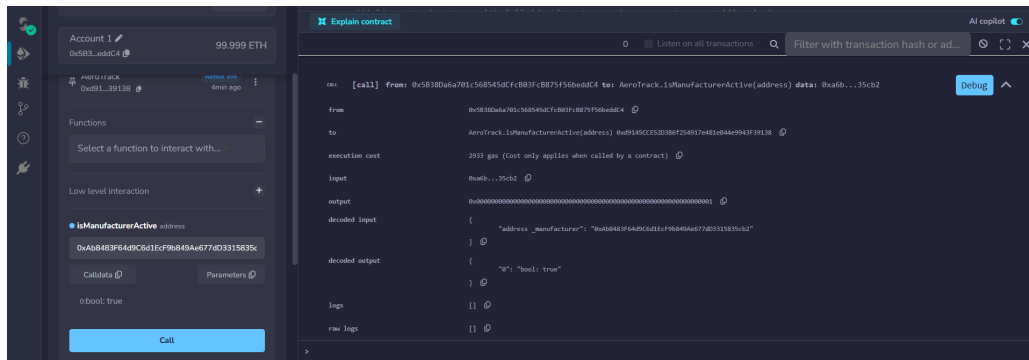


Figure 28: isManufacturerActive returns true for registered manufacturer

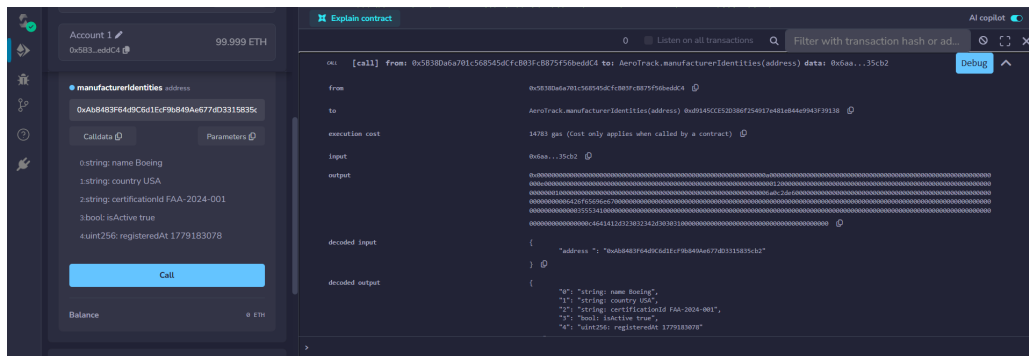


Figure 29: Public verification of manufacturer identity on-chain

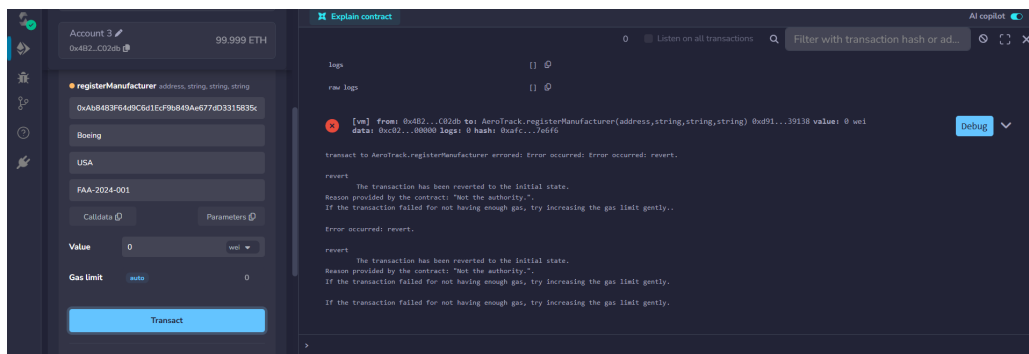


Figure 30: Unauthorized registration attempt fails (only authority can register)

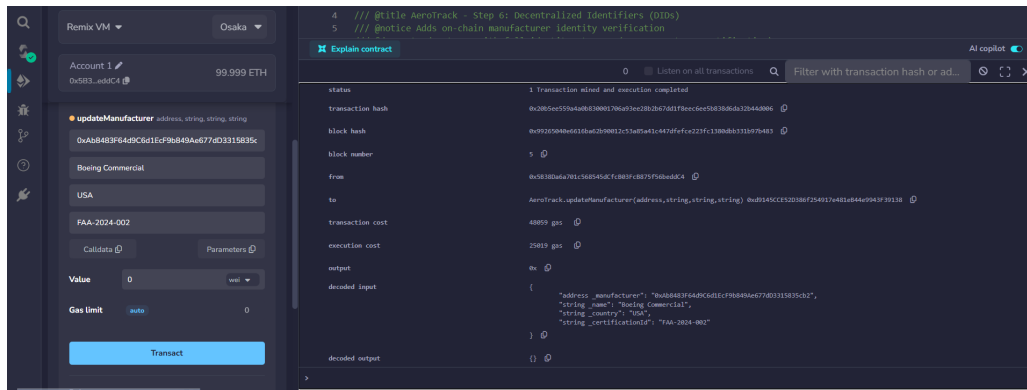


Figure 31: Authority updates manufacturer identity information

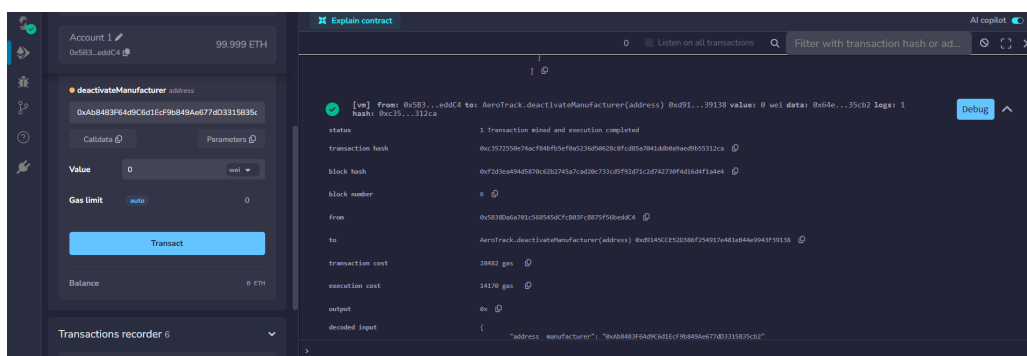


Figure 32: Authority deactivates manufacturer (DID revocation)

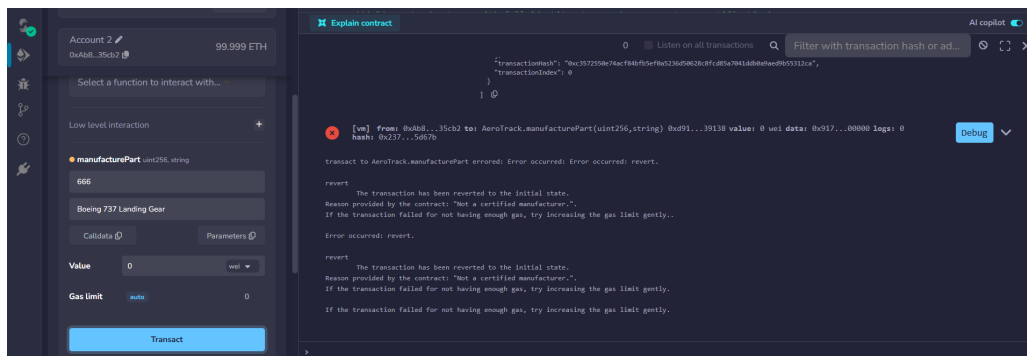


Figure 33: Deactivated manufacturer fails to create parts

3.7 Step 7: IoT Oracle Simulation

To address the **IoT Oracles** research topic, we implemented an authorized sensor system. Physical IoT devices (simulated by Remix accounts) can automatically report maintenance data for their assigned parts without human intervention.

```
1 struct Sensor {
2     uint256 assignedPart;    // Which part this sensor monitors
3     string sensorType;      // "temperature", "vibration", "stress"
4     bool isActive;
5     uint256 registeredAt;
```

```

6 }
7
8 mapping(address => Sensor) public sensors;
9
10 modifier onlyActiveSensor() {
11     require(sensors[msg.sender].isActive, "Not an authorized sensor.");
12     _;
13 }
14
15 function registerSensor(address _sensor, uint256 _sn,
16     string memory _type) public onlyAuthority partExists(_sn) {
17     sensors[_sensor] = Sensor(_sn, _type, true, block.timestamp);
18     emit SensorRegistered(_sensor, _sn, _type);
19 }
20
21 function logSensorData(string memory _dataType,
22     string memory _value) public onlyActiveSensor {
23     uint256 sn = sensors[msg.sender].assignedPart;
24     emit SensorDataLogged(sn, msg.sender, _dataType, _value);
25 }

```

Listing 10: IoT Sensor struct and registration

The sensor does not pass the serial number — it is automatically resolved from the registry. This prevents a sensor from reporting data for parts it is not assigned to.

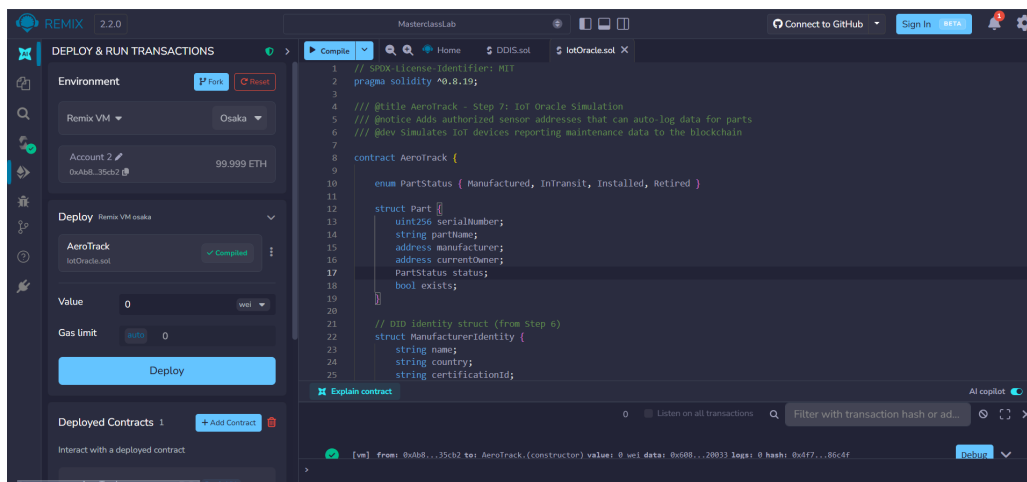


Figure 34: IoT Oracle contract code in Remix

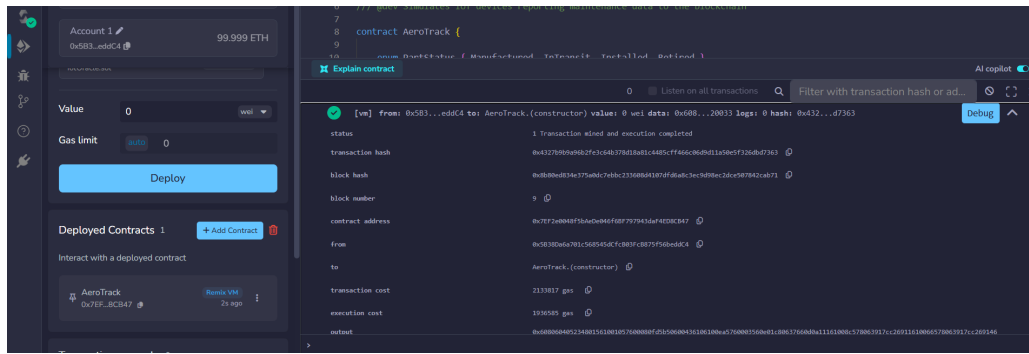


Figure 35: Successful deployment of the IoT-enhanced contract

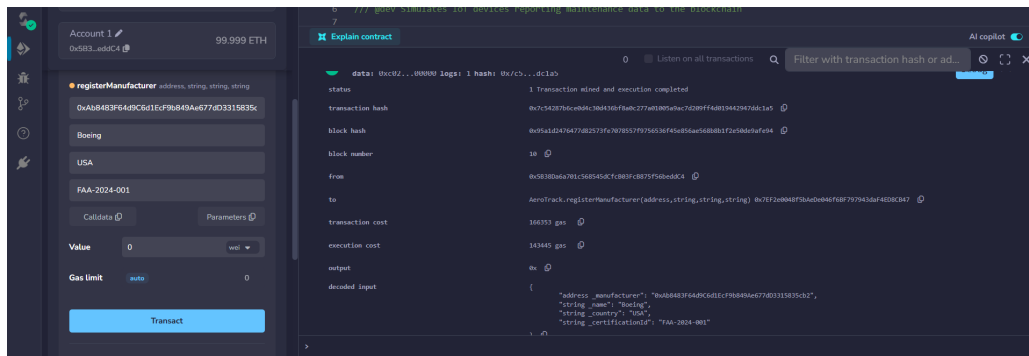


Figure 36: Registering Boeing as a certified manufacturer

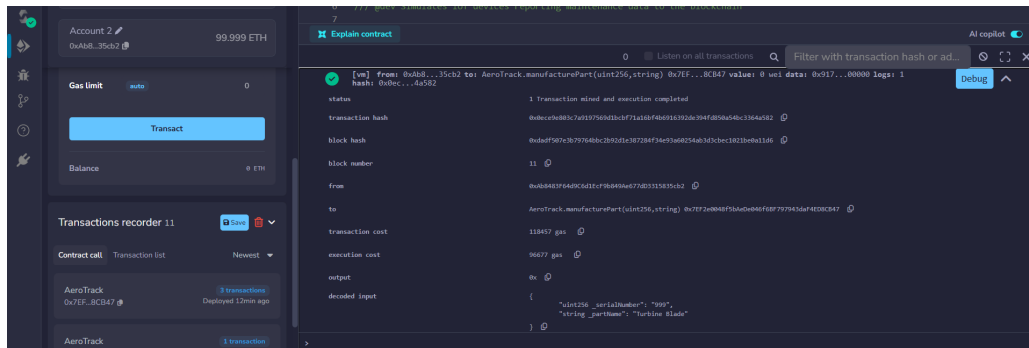


Figure 37: Account 2 (Boeing) manufactures part 999

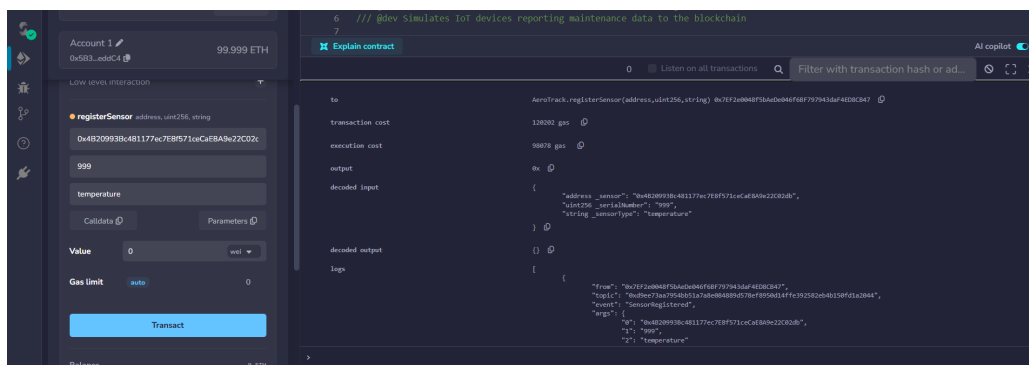


Figure 38: Authority registers Account 3 as a temperature sensor for part 999

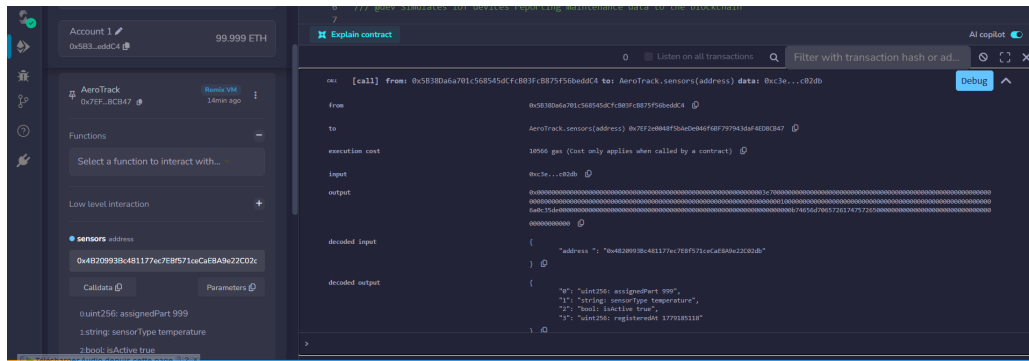


Figure 39: Querying sensors shows assigned part and sensor type

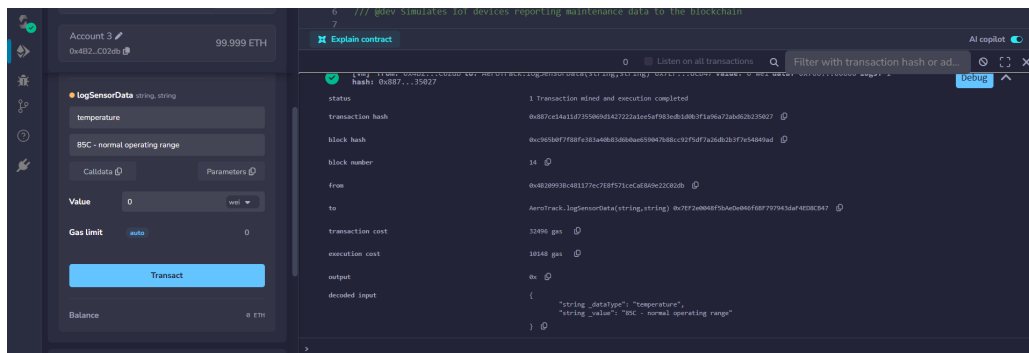


Figure 40: Sensor reports temperature reading via logSensorData

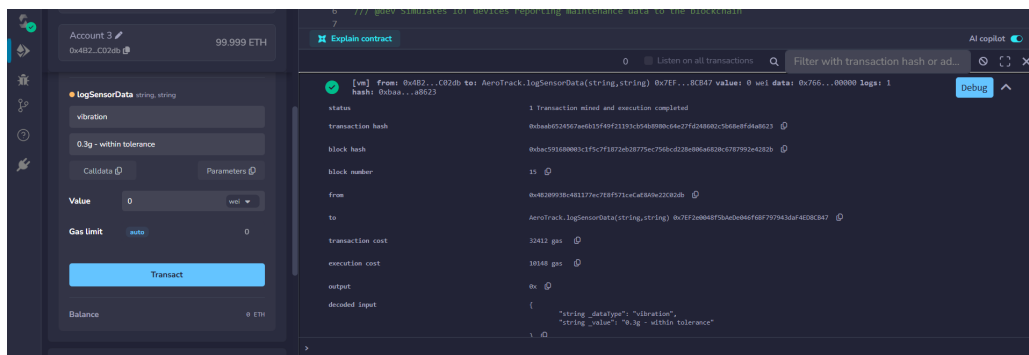


Figure 41: Sensor reports vibration reading

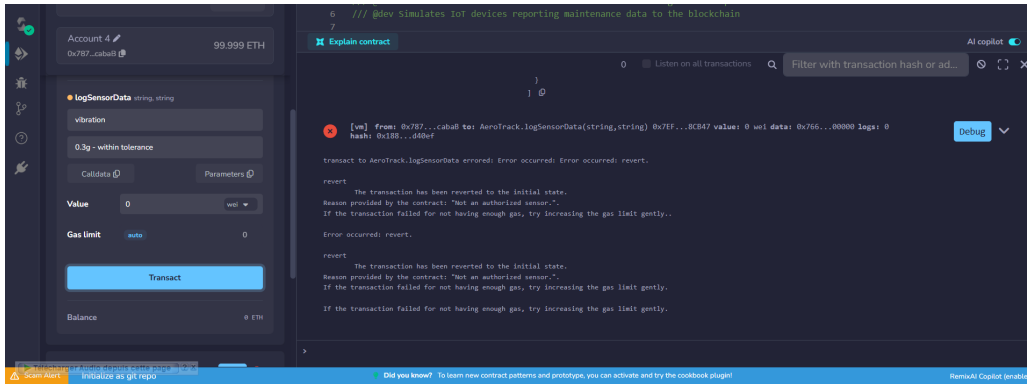


Figure 42: Unauthorized Account 2 fails to log sensor data

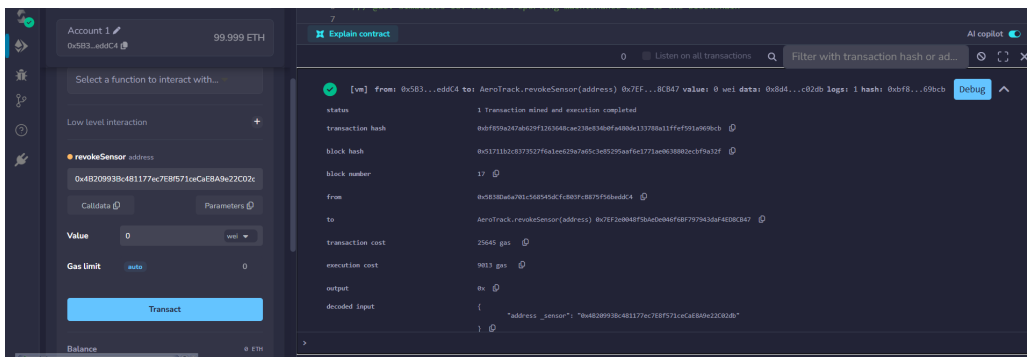


Figure 43: Authority revokes sensor authorization from Account 3

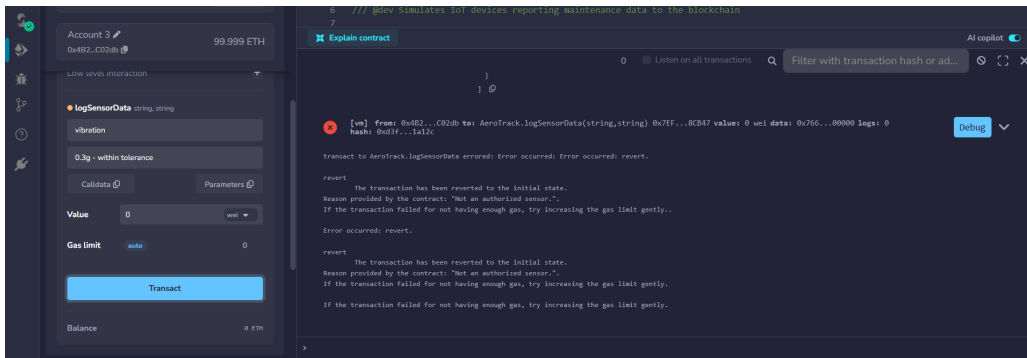


Figure 44: Revoked sensor (Account 3) fails to report data

3.8 Step 8: Real IoT Oracle with MQTT Broker

To demonstrate a production-ready architecture, we implemented a full IoT pipeline using **MQTT** (Message Queuing Telemetry Transport) as the communication protocol between sensors and the blockchain. The system uses Docker containers for the MQTT broker (Mosquitto) and a local Ethereum node (Ganache).

The architecture follows the oracle pattern:

1. A Python sensor simulator publishes temperature readings to the MQTT broker
2. A Node.js bridge subscribes to the MQTT topic

- When data arrives, the bridge signs a transaction and calls `logSensorData()` on the smart contract
- The blockchain permanently records the sensor reading as an event

```

C:\Users\Noaman\Desktop\Ensem\second year\S4\BlockchainTools\Final lab\iot-oracle>docker-compose up -d
time="2026-05-19T11:31:19+01:00" level=warning msg="C:\Users\Noaman\Desktop\Ensem\second year\S4\BlockchainTools\Final lab\iot-oracle\docker-compo
se.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 8/8
✓Image eclipse-mosquito:2 Pulled 11.1s
✓Network iot-oracle_default Created 0.1s
✓Container aerotrack-ganache Created 0.6s
✓Container aerotrack-mqtt Created 0.6s

C:\Users\Noaman\Desktop\Ensem\second year\S4\BlockchainTools\Final lab\iot-oracle>docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
689ff3ee316d   eclipse-mosquito:2                  "/docker-entrypoint..." 22 seconds ago Up 20 seconds 0.0.0.0:1883->1883/tcp, [::]:1883->1883/tcp, 0.0.0.0:
9001->9001/tcp, [::]:9001->9001/tcp
dbellab9f3bf   trufflesuite/ganache:latest        "node /app/dist/node..." 22 seconds ago Up 20 seconds 0.0.0.0:8545->8545/tcp, [::]:8545->8545/tcp

C:\Users\Noaman\Desktop\Ensem\second year\S4\BlockchainTools\Final lab\iot-oracle>

```

Figure 45: Contract deployed on Ganache via Remix (External Http Provider)

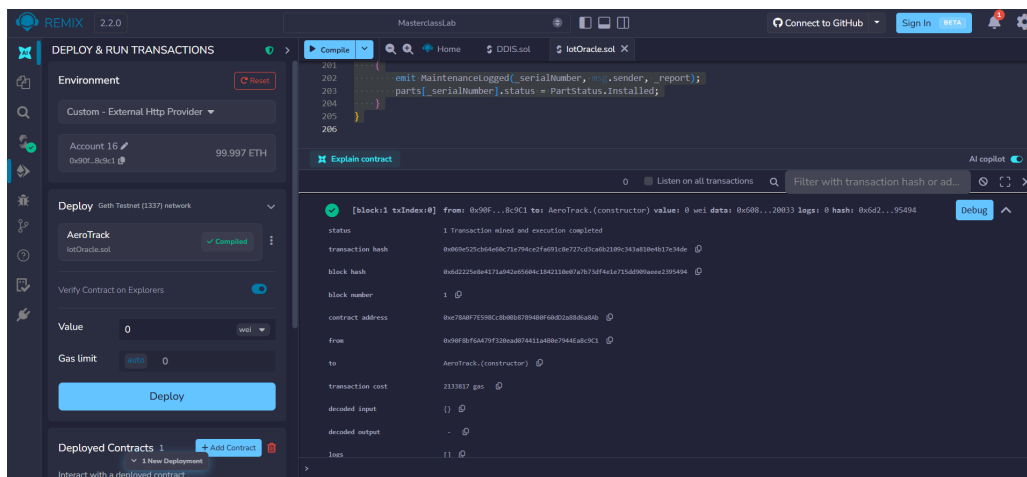


Figure 46: Registering manufacturer on Ganache blockchain

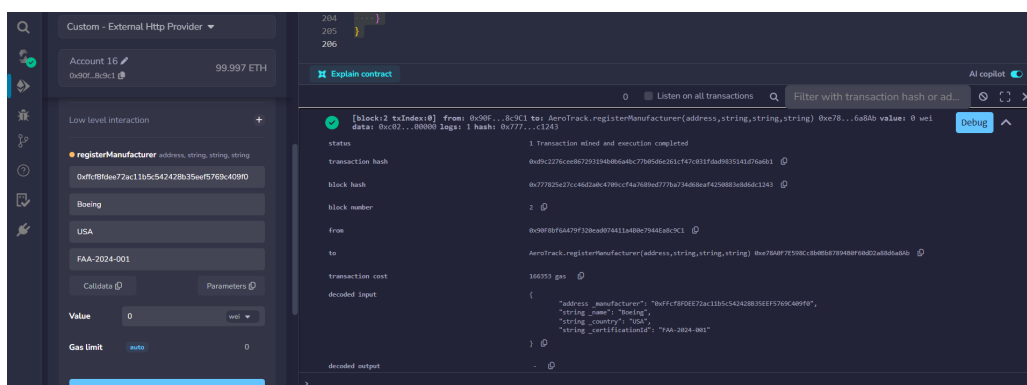


Figure 47: Manufacturing part 999 on Ganache

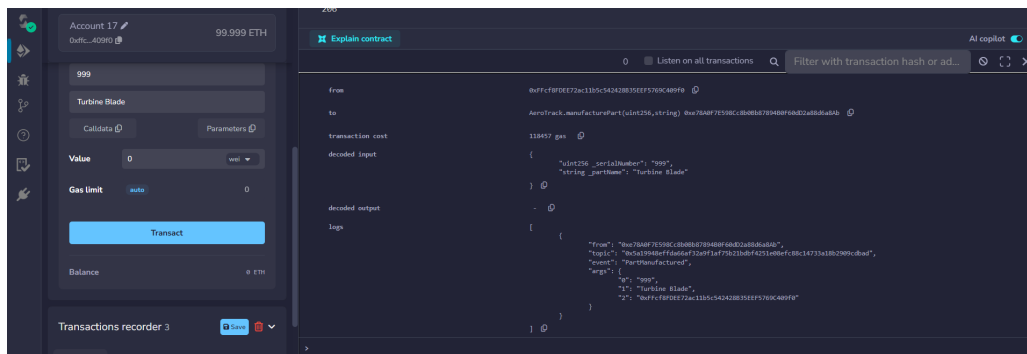


Figure 48: Registering IoT sensor address on Ganache

```
C:\Users\Woaman\Desktop\Ensem\second year\S4\BlockchainTools\Final lab\iot-oracle>cd bridge
C:\Users\Woaman\Desktop\Ensem\second year\S4\BlockchainTools\Final lab\iot-oracle\bridge>npm install
up to date, audited 48 packages in 4s
9 packages are looking for funding
  run 'npm fund' for details
2 moderate severity vulnerabilities
To address all issues (including breaking changes), run:
  npm audit fix --force
Run 'npm audit' for details.
C:\Users\Woaman\Desktop\Ensem\second year\S4\BlockchainTools\Final lab\iot-oracle\bridge>npm run deploy
> aerotrack-iot-bridge@1.0.0 deploy
> node deploy.js
=====
AeroTrack - Contract Deployment to Ganache
=====
[OK] Connected to Ganache (chainId: 1337)
[OK] Found 10 accounts
  Authority: 0x90F8Bf6A4779f232ead077411a4B0e7944Ea8c9C1
  Manufacturer: 0xFFc8f8DEE72ac11b5c542428B3EEF5769C4094f0
  Sensor: 0x22d491Bde230f2f4325b2108D26f1eAbA1e32b
[OK] Config saved to config.json
```

Figure 49: Node.js bridge setup: npm install and deploy helper

[illegible]

Figure 50: Bridge running: relaying MQTT messages to blockchain transactions

```

C:\Users\Noaman\Desktop\Ensem\second_year\S4\BlockchainTools\Final lab\iot-oracle\sensor>python sensor_simulator.py
=====
AeroTrack IoT Sensor Simulator
Turbine Blade Temperature Sensor
=====
[SENSOR] Connected to MQTT broker (code: Success)
[SENSOR] Publishing to topic: aerotrack/sensors/part999
[SENSOR] Sensor ID: SENSOR-TEMP-001 | Part: 999
=====
[SENSOR] Starting readings (every 5 seconds)...
[SENSOR] Press Ctrl+C to stop

[1] ✓ 82.6C (NORMAL) -> aerotrack/sensors/part999
[2] ✓ 86.1C (NORMAL) -> aerotrack/sensors/part999
[3] ✓ 74.5C (NORMAL) -> aerotrack/sensors/part999
[4] ✓ 70.4C (NORMAL) -> aerotrack/sensors/part999
[5] ✓ 85.0C (NORMAL) -> aerotrack/sensors/part999

```

Figure 51: Python sensor simulator publishing temperature readings via MQTT

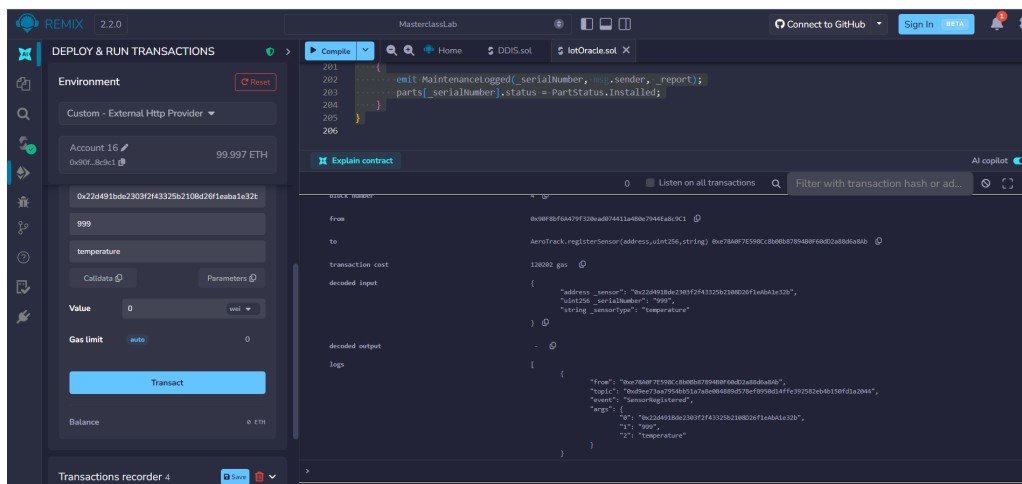


Figure 52: Remix showing SensorDataLogged events from automated MQTT pipeline

4 Conclusion

This project demonstrated how **Ethereum smart contracts** can secure the provenance of airplane spare parts. The AeroTrack contract implements:

- **Immutable registration** of manufactured parts
- **Role-based access control** via Solidity modifiers
- **Ownership tracking** through the supply chain
- **Gas-efficient traceability** using events

References

1. Ethereum Foundation, “Solidity Documentation,” <https://docs.soliditylang.org/>
2. Remix IDE, <https://remix.ethereum.org/>

3. EASA, “Airworthiness Directives and Certificates,” <https://www.easa.europa.eu/>

A Complete Source Code

A.1 Step 1–4: Base AeroTrack Contract

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.19;
3
4 contract AeroTrack {
5
6     enum PartStatus { Manufactured, InTransit, Installed, Retired }
7
8     struct Part {
9         uint256 serialNumber;
10        string partName;
11        address manufacturer;
12        address currentOwner;
13        PartStatus status;
14        bool exists;
15    }
16
17    address public regulatoryAuthority;
18    mapping(uint256 => Part) public parts;
19
20    event PartManufactured(uint256 indexed serialNumber, string
21        partName, address manufacturer);
22    event OwnershipTransferred(uint256 indexed serialNumber, address
23        oldOwner, address newOwner);
24    event MaintenanceLogged(uint256 indexed serialNumber, address
25        mechanic, string report);
26
27    constructor() {
28        regulatoryAuthority = msg.sender;
29    }
30
31    modifier partExists(uint256 _serialNumber) {
32        require(parts[_serialNumber].exists == true, "Part does not
33            exist.");
34        _;
35    }
36
37    modifier onlyPartOwner(uint256 _serialNumber) {
38        require(parts[_serialNumber].currentOwner == msg.sender, "Not
39            the owner.");
40        _;
41    }
42
43    function manufacturePart(uint256 _serialNumber, string memory
44        _partName) public {
45        require(parts[_serialNumber].exists == false, "Serial Number
46            taken!");
47        parts[_serialNumber] = Part({
48            serialNumber: _serialNumber,
49            partName: _partName,
50            manufacturer: msg.sender,
51            currentOwner: msg.sender,
```

```

45         status: PartStatus.Manufactured,
46         exists: true
47     });
48     emit PartManufactured(_serialNumber, _partName, msg.sender);
49 }
50
51 function transferPart(uint256 _serialNumber, address _newOwner)
52     public partExists(_serialNumber) onlyPartOwner(_serialNumber)
53 {
54     require(_newOwner != address(0), "Invalid address.");
55     address oldOwner = parts[_serialNumber].currentOwner;
56     parts[_serialNumber].currentOwner = _newOwner;
57     parts[_serialNumber].status = PartStatus.InTransit;
58     emit OwnershipTransferred(_serialNumber, oldOwner, _newOwner)
59     ;
60 }
61
62 function logMaintenance(uint256 _serialNumber, string memory
63     _report)
64     public partExists(_serialNumber) onlyPartOwner(_serialNumber)
65 {
66     emit MaintenanceLogged(_serialNumber, msg.sender, _report);
67     parts[_serialNumber].status = PartStatus.Installed;
68 }

```

Listing 11: AeroTrack.sol – Complete base contract

A.2 Step 5: Certified Manufacturer Registry

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.19;
3
4 contract AeroTrack {
5
6     enum PartStatus { Manufactured, InTransit, Installed, Retired }
7
8     struct Part {
9         uint256 serialNumber;
10        string partName;
11        address manufacturer;
12        address currentOwner;
13        PartStatus status;
14        bool exists;
15    }
16
17    address public regulatoryAuthority;
18    mapping(uint256 => Part) public parts;
19    mapping(address => bool) public certifiedManufacturers;
20
21    event PartManufactured(uint256 indexed serialNumber, string
22        partName, address manufacturer);
23    event OwnershipTransferred(uint256 indexed serialNumber, address
24        oldOwner, address newOwner);
25    event MaintenanceLogged(uint256 indexed serialNumber, address
26        mechanic, string report);

```

```

24     event ManufacturerCertified(address indexed manufacturer);
25     event ManufacturerRevoked(address indexed manufacturer);
26
27     constructor() { regulatoryAuthority = msg.sender; }
28
29     modifier partExists(uint256 _serialNumber) {
30         require(parts[_serialNumber].exists == true, "Part does not
31             exist.");
32     }
33     modifier onlyPartOwner(uint256 _serialNumber) {
34         require(parts[_serialNumber].currentOwner == msg.sender, "Not
35             the owner.");
36     }
37     modifier onlyAuthority() {
38         require(msg.sender == regulatoryAuthority, "Not the authority
39             .");
40     }
41     modifier onlyCertified() {
42         require(certifiedManufacturers[msg.sender], "Not a certified
43             manufacturer.");
44     }
45
46     function certifyManufacturer(address _manufacturer) public
47         onlyAuthority {
48         require(_manufacturer != address(0), "Invalid address.");
49         certifiedManufacturers[_manufacturer] = true;
50         emit ManufacturerCertified(_manufacturer);
51     }
52
53     function revokeManufacturer(address _manufacturer) public
54         onlyAuthority {
55         certifiedManufacturers[_manufacturer] = false;
56         emit ManufacturerRevoked(_manufacturer);
57     }
58
59     function manufacturePart(uint256 _serialNumber, string memory
60         _partName)
61         public onlyCertified
62     {
63         require(parts[_serialNumber].exists == false, "Serial Number
64             taken!");
65         parts[_serialNumber] = Part({
66             serialNumber: _serialNumber, partName: _partName,
67             manufacturer: msg.sender, currentOwner: msg.sender,
68             status: PartStatus.Manufactured, exists: true
69         });
70         emit PartManufactured(_serialNumber, _partName, msg.sender);
71     }
72
73     function transferPart(uint256 _serialNumber, address _newOwner)
74         public partExists(_serialNumber) onlyPartOwner(_serialNumber)
75     {
76         require(_newOwner != address(0), "Invalid address.");
77         address oldOwner = parts[_serialNumber].currentOwner;

```

```

74     parts[_serialNumber].currentOwner = _newOwner;
75     parts[_serialNumber].status = PartStatus.InTransit;
76     emit OwnershipTransferred(_serialNumber, oldOwner, _newOwner)
77     ;
78 }
79 function logMaintenance(uint256 _serialNumber, string memory
80     _report)
81     public partExists(_serialNumber) onlyPartOwner(_serialNumber)
82     {
83         emit MaintenanceLogged(_serialNumber, msg.sender, _report);
84         parts[_serialNumber].status = PartStatus.Installed;
85     }
86 }

```

Listing 12: Step5_Enhancement.sol – Adds certification system

A.3 Step 6: Decentralized Identifiers (DIDs)

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.19;
3
4 contract AeroTrack {
5
6     enum PartStatus { Manufactured, InTransit, Installed, Retired }
7
8     struct Part {
9         uint256 serialNumber; string partName;
10        address manufacturer; address currentOwner;
11        PartStatus status; bool exists;
12    }
13
14    struct ManufacturerIdentity {
15        string name; string country; string certificationId;
16        bool isActive; uint256 registeredAt;
17    }
18
19    address public regulatoryAuthority;
20    mapping(uint256 => Part) public parts;
21    mapping(address => bool) public certifiedManufacturers;
22    mapping(address => ManufacturerIdentity) public
23        manufacturerIdentities;
24
25    event PartManufactured(uint256 indexed serialNumber, string
26        partName, address manufacturer);
27    event OwnershipTransferred(uint256 indexed serialNumber, address
28        oldOwner, address newOwner);
29    event MaintenanceLogged(uint256 indexed serialNumber, address
30        mechanic, string report);
31    event ManufacturerRegistered(address indexed manufacturer, string
32        name, string certificationId);
33    event ManufacturerDeactivated(address indexed manufacturer);
34
35    constructor() { regulatoryAuthority = msg.sender; }
36 }

```

```

32     modifier partExists(uint256 _sn) { require(parts[_sn].exists, "
        Part does not exist."); _; }
33     modifier onlyPartOwner(uint256 _sn) { require(parts[_sn].
        currentOwner == msg.sender, "Not the owner."); _; }
34     modifier onlyAuthority() { require(msg.sender ==
        regulatoryAuthority, "Not the authority."); _; }
35     modifier onlyCertified() { require(certifiedManufacturers[msg.
        sender], "Not certified."); _; }
36
37     function registerManufacturer(address _m, string memory _name,
38         string memory _country, string memory _certId) public
        onlyAuthority {
39         manufacturerIdentities[_m] = ManufacturerIdentity(_name,
        _country, _certId, true, block.timestamp);
40         certifiedManufacturers[_m] = true;
41         emit ManufacturerRegistered(_m, _name, _certId);
42     }
43
44     function deactivateManufacturer(address _m) public onlyAuthority
        {
45         manufacturerIdentities[_m].isActive = false;
46         certifiedManufacturers[_m] = false;
47         emit ManufacturerDeactivated(_m);
48     }
49
50     function isManufacturerActive(address _m) public view returns (
        bool) {
51         return manufacturerIdentities[_m].isActive;
52     }
53
54     function manufacturePart(uint256 _sn, string memory _name) public
        onlyCertified {
55         require(!parts[_sn].exists, "Serial Number taken!");
56         parts[_sn] = Part(_sn, _name, msg.sender, msg.sender,
        PartStatus.Manufactured, true);
57         emit PartManufactured(_sn, _name, msg.sender);
58     }
59
60     function transferPart(uint256 _sn, address _new)
        public partExists(_sn) onlyPartOwner(_sn) {
61         require(_new != address(0), "Invalid address.");
62         address old = parts[_sn].currentOwner;
63         parts[_sn].currentOwner = _new;
64         parts[_sn].status = PartStatus.InTransit;
65         emit OwnershipTransferred(_sn, old, _new);
66     }
67
68
69     function logMaintenance(uint256 _sn, string memory _report)
        public partExists(_sn) onlyPartOwner(_sn) {
70         emit MaintenanceLogged(_sn, msg.sender, _report);
71         parts[_sn].status = PartStatus.Installed;
72     }
73 }
74 }

```

Listing 13: Step6_DIDs.sol – Adds on-chain identity

A.4 Step 7: IoT Oracle Simulation

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.19;
3
4 contract AeroTrack {
5
6     enum PartStatus { Manufactured, InTransit, Installed, Retired }
7
8     struct Part {
9         uint256 serialNumber; string partName;
10        address manufacturer; address currentOwner;
11        PartStatus status; bool exists;
12    }
13
14    struct ManufacturerIdentity {
15        string name; string country; string certificationId;
16        bool isActive; uint256 registeredAt;
17    }
18
19    struct Sensor {
20        uint256 assignedPart; string sensorType;
21        bool isActive; uint256 registeredAt;
22    }
23
24    address public regulatoryAuthority;
25    mapping(uint256 => Part) public parts;
26    mapping(address => bool) public certifiedManufacturers;
27    mapping(address => ManufacturerIdentity) public
28        manufacturerIdentities;
29    mapping(address => Sensor) public sensors;
30
31    event PartManufactured(uint256 indexed serialNumber, string
32        partName, address manufacturer);
33    event OwnershipTransferred(uint256 indexed serialNumber, address
34        oldOwner, address newOwner);
35    event MaintenanceLogged(uint256 indexed serialNumber, address
36        mechanic, string report);
37    event ManufacturerRegistered(address indexed manufacturer, string
38        name, string certificationId);
39    event ManufacturerDeactivated(address indexed manufacturer);
40    event SensorRegistered(address indexed sensor, uint256 indexed
41        serialNumber, string sensorType);
42    event SensorRevoked(address indexed sensor);
43    event SensorDataLogged(uint256 indexed serialNumber, address
44        indexed sensor, string dataType, string value);
45
46    constructor() { regulatoryAuthority = msg.sender; }
47
48    modifier partExists(uint256 _sn) { require(parts[_sn].exists, "
49        Part does not exist."); _; }
50    modifier onlyPartOwner(uint256 _sn) { require(parts[_sn].
51        currentOwner == msg.sender, "Not the owner."); _; }
52    modifier onlyAuthority() { require(msg.sender ==
53        regulatoryAuthority, "Not the authority."); _; }
54    modifier onlyCertified() { require(certifiedManufacturers[msg.
55        sender], "Not certified."); _; }

```

```

45     modifier onlyActiveSensor() { require(sensors[msg.sender].
46         isActive, "Not an authorized sensor."); _; }
47
48     function registerManufacturer(address _m, string memory _name,
49         string memory _country, string memory _certId) public
50         onlyAuthority {
51         manufacturerIdentities[_m] = ManufacturerIdentity(_name,
52             _country, _certId, true, block.timestamp);
53         certifiedManufacturers[_m] = true;
54         emit ManufacturerRegistered(_m, _name, _certId);
55     }
56
57     function deactivateManufacturer(address _m) public onlyAuthority
58     {
59         manufacturerIdentities[_m].isActive = false;
60         certifiedManufacturers[_m] = false;
61         emit ManufacturerDeactivated(_m);
62     }
63
64     function registerSensor(address _sensor, uint256 _sn, string
65         memory _type)
66         public onlyAuthority partExists(_sn) {
67         require(_sensor != address(0), "Invalid sensor address.");
68         sensors[_sensor] = Sensor(_sn, _type, true, block.timestamp);
69         emit SensorRegistered(_sensor, _sn, _type);
70     }
71
72     function revokeSensor(address _sensor) public onlyAuthority {
73         sensors[_sensor].isActive = false;
74         emit SensorRevoked(_sensor);
75     }
76
77     function logSensorData(string memory _dataType, string memory
78         _value)
79         public onlyActiveSensor {
80         uint256 sn = sensors[msg.sender].assignedPart;
81         emit SensorDataLogged(sn, msg.sender, _dataType, _value);
82     }
83
84     function isSensorActive(address _sensor) public view returns (
85         bool) {
86         return sensors[_sensor].isActive;
87     }
88
89     function manufacturePart(uint256 _sn, string memory _name) public
90         onlyCertified {
91         require(!parts[_sn].exists, "Serial Number taken!");
92         parts[_sn] = Part(_sn, _name, msg.sender, msg.sender,
93             PartStatus.Manufactured, true);
94         emit PartManufactured(_sn, _name, msg.sender);
95     }
96
97     function transferPart(uint256 _sn, address _new)
98         public partExists(_sn) onlyPartOwner(_sn) {
99         require(_new != address(0), "Invalid address.");
100        address old = parts[_sn].currentOwner;
101        parts[_sn].currentOwner = _new;
102        parts[_sn].status = PartStatus.InTransit;

```



```
94         emit OwnershipTransferred(_sn, old, _new);
95     }
96
97     function logMaintenance(uint256 _sn, string memory _report)
98     public partExists(_sn) onlyPartOwner(_sn) {
99         emit MaintenanceLogged(_sn, msg.sender, _report);
100         parts[_sn].status = PartStatus.Installed;
101     }
102 }
```

Listing 14: Step7_IoTOracle.sol – Adds IoT sensor system